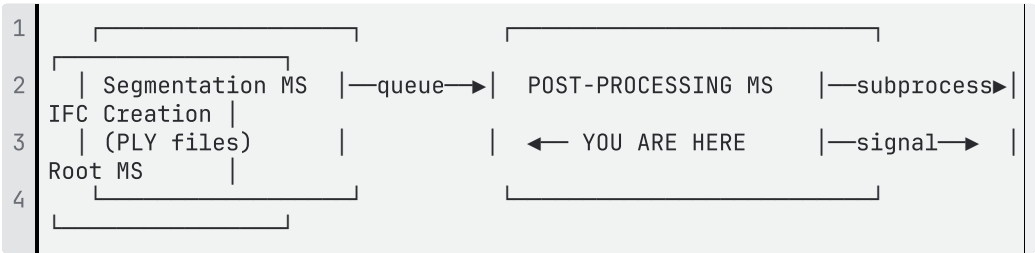


IFC Post-Processing Microservice Architecture and Flow

The Post-Processing Microservice sits immediately after Segmentation and before workflow completion. It converts segmented point-cloud outputs into structured building and equipment geometry, enriches rack content using image-based detection, resolves spatial conflicts, creates the IFC deliverable through an embedded .NET engine, uploads the result, and signals completion back to the orchestration layer.

 This page documents the **Post-Processing Microservice** only. It begins after Segmentation has already produced class-wise PLY outputs and extracted images, and it ends once the IFC has been generated, uploaded, and completion has been signaled.

Position in the pipeline



Direction	Service	Interaction
Upstream	Segmentation MS	Dispatches segmented PLY artifacts and extracted images through the post-processing queue
Downstream	IFC Creation Engine	Embedded .NET subprocess invoked through <code>IFCCreator.dll</code>
Downstream	Root MS	Receives completion signal after successful artifact generation and upload

Service overview

This service transforms segmented geometry into semantically organized output that can be serialized into IFC. It is the stage where raw semantic segmentation outputs become structured, conflict-resolved, and delivery-ready building objects.

Property	Value
Primary language	Python
Runtime mode	Long-running queue consumer
Health endpoint	Port <code>4000</code> in a separate health process
Queue	Azure Service Bus post-processing queue
Storage	Azure Blob storage for segmented inputs, images, and backup IFC upload
IFC generation	.NET DLL subprocess using <code>IFCCreator.dll</code>
Database	MongoDB for job tracking
Image enrichment	YOLO-based equipment detection
Memory profile	High-memory processing workload with dedicated runtime limits

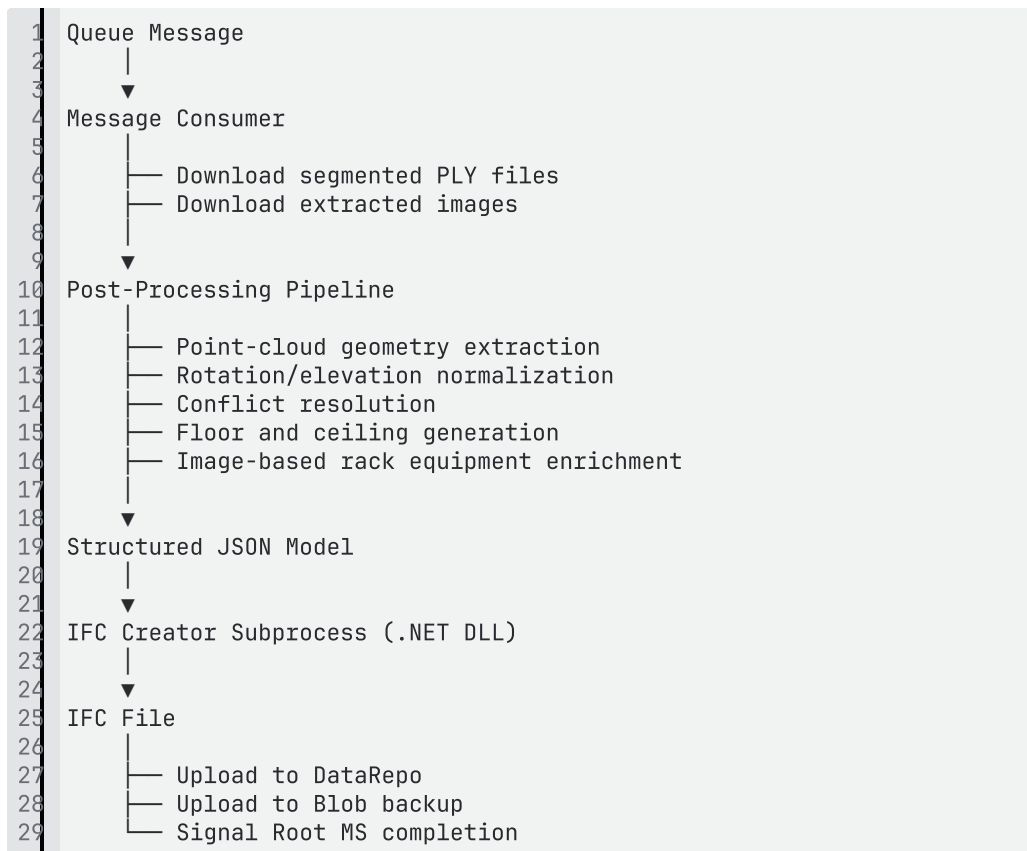
What this microservice is responsible for

- Consume post-segmentation processing requests from the queue
- Download segmented PLY files and image artifacts from blob storage
- Extract geometric entities from per-class point clouds
- Infer structured objects such as walls, doors, racks, cabinets, rectifiers, batteries, floors, and ceilings
- Apply rotation correction and elevation normalization where required
- Resolve equipment-to-equipment and equipment-to-wall spatial conflicts
- Run YOLO-based image detection to enrich rack equipment information

- Serialize the extracted geometry into a JSON payload for IFC generation
- Invoke the IFC creation engine through a subprocess
- Upload the final IFC to external storage targets
- Signal workflow completion to the Root orchestration service

i Conceptually, Segmentation answers **what points belong to which class**. Post-Processing answers **what physical objects exist, where they are, how they relate spatially, and how they should be represented in IFC**.

High-level architecture



Module structure

```

1 app/
2 |— main.py                # Entry: queue consumer + health
  server
3 |— queue/
4 |   |— message_consumer.py # Service Bus consumer and
  orchestration
5 |   |— message_producer.py  # Completion messaging support
6 |— service/
7 |   |— post_processing_service.py # End-to-end pipeline
  orchestration
8 |   |— point_cloud_processing.py # Geometry extraction from
  segmented PLY files
9 |   |— image_processing.py     # YOLO-based image enrichment
10 |— components/
  
```

```

11 |   |─ wall.py           # Wall extraction
12 |   |─ door.py          # Door extraction and wall
    association
13 |   |─ rack.py          # Rack extraction and internal
    equipment grouping
14 |   |─ cabinet.py       # Cabinet extraction
15 |   |─ battery_cabinet.py # Battery cabinet extraction
16 |   |─ battery.py       # Battery extraction relative to
    parent cabinet
17 |   |─ rectifier.py     # Rectifier extraction relative to
    parent cabinet
18 |   |─ floor_ceiling.py # Floor/ceiling slab generation
19 |─ ifc_service/
20 |   |─ ifc_creator.py   # Python wrapper for .NET invocation
21 |   |─ IFCCreator.dll   # IFC generation engine
22 |─ geometry/           # Shared geometric utilities
23 |─ ml_models/          # YOLO model assets
24 |─ helper/
25 |   |─ blob_helper.py   # Blob download/upload
26 |   |─ cognito_helper.py # Auth for DataRepo upload flow
27 |   |─ conflict_resolver.py # Spatial conflict resolution
28 |   |─ retry_helper.py  # Retry and backoff helpers
29 |─ api/
30 |   |─ get/upload_sas.py # Request SAS upload URL
31 |   |─ post/upload_ifc.py # Upload final IFC
32 |─ config/config_reader.py # Env and config.ini reader
33 |─ database/jobs_repo.py  # MongoDB job state tracking
34 |─ schemas/message_schema.py # Pydantic message validation
35 |─ temporal/client.py    # Temporal progress signaling
36 |─ Properties/Property.py # Project and site metadata
    extraction
37 |─ utils/                # Rotation, elevation, wall, and
    related helpers

```

End-to-end processing flow

Entry and orchestration

```

1 | main.py
2 |
3 | └─ Health process
4 |   └─ /health
5 |
6 | └─ Queue consumer
7 |   └─ Receive message
8 |     └─ Validate payload
9 |     └─ Download PLY artifacts
10 |     └─ Download images
11 |     └─ Run post_processing_main(...)
12 |     └─ Upload IFC to DataRepo
13 |     └─ Upload IFC to Blob backup
14 |     └─ Signal Root MS
15 |     └─ Complete message

```

Main service pipeline

```

1 | post_processing_main(input_path, meta_data)
2 |
3 | └─ 1. process_point_cloud(...)
4 | └─ 2. resolve_equipment_conflicts(...)
5 | └─ 3. resolve_equipment_wall_conflicts(...)
6 | └─ 4. FloorCeiling.extract_geometry(...)
7 | └─ 5. image_processing_main(...)

```

```

8 | 6. check_rack_equipment_overlap_and_adjust(...)
9 | 7. Write structured JSON
10 | 8. IFCCreator.create_ifc(...)

```

Step	Purpose
1	Extract core geometry from segmented PLY files and assemble the initial structured data model
2	Resolve overlaps across equipment categories so objects occupy physically plausible space
3	Prevent equipment from intruding into wall planes
4	Generate floor and ceiling slab geometry from wall bounds and elevation context
5	Enrich rack contents using YOLO detections from extracted images
6	Adjust final equipment layout within racks to reduce internal overlap
7	Persist the complete object graph to JSON for downstream IFC conversion
8	Invoke the .NET IFC creator and generate the final deliverable file

Point-cloud geometry extraction

The geometric extraction stage is the core of the service. It interprets class-separated PLY files and converts them into structured entities with dimensions, identity, orientation, and spatial relationships.

```

1 | process_point_cloud()
2 |
3 |   1. Floor elevation extraction
4 |   2. Wall extraction
5 |   3. Cabinet extraction
6 |   4. Battery cabinet extraction
7 |   5. Door extraction
8 |   6. Rack extraction
9 |   7. Rectifier extraction
10 |   8. Battery extraction
11 |   9. Property extraction
12 |  10. Height adjustment

```

Stage	Input	Core logic	Output
Elevation	Floor-related point cloud	Estimate floor and ceiling reference values	[floor_z, ceiling_z]
Wall	Wall PLY + elevation	Project to XY, detect contours, fit lines, connect segments, estimate width and rotation	Wall segments with footprint and height
Cabinet	Cabinet PLY + elevation	Cluster instances and fit oriented bounding boxes	Cabinet objects
Battery cabinet	Battery cabinet PLY + elevation	Cluster and fit geometry similar to cabinet flow	Battery cabinet objects
Door	Door PLY + wall geometry	Associate clustered door points with containing wall planes	Door objects linked to walls
Rack	Rack PLY + elevation	Cluster racks, fit OBBs, detect structural edges, group internal equipment by Z	Rack objects with equipment slots
Rectifier	Rectifier PLY + cabinet context	Assign to nearest parent cabinet and extract dimensions	Rectifier objects with parent links
Battery	Battery PLY + cabinet context	Assign to nearest parent cabinet and extract dimensions	Battery objects with parent links
Property	Metadata payload	Map project and site metadata into structured properties	Project and site property sets

Height adjustment	All extracted geometry	Extend wall height when detected equipment exceeds nominal ceiling	Consistent vertical enclosure
-------------------	------------------------	--	-------------------------------

Component extraction logic

Wall extraction

```

1 Load wall PLY
2   → rotation correction / axis alignment check
3   → XY projection
4   → 2D histogram
5   → edge detection
6   → contour extraction
7   → line fitting
8   → segment connection
9   → width estimation
10  → wall geometry output

```

Wall extraction establishes the envelope used by several later steps. It not only creates wall geometry, but also provides a spatial frame for door placement, floor/ceiling generation, and wall-conflict resolution.

Door extraction

```

1 Load door points
2   → identify containing wall
3   → cluster points
4   → estimate dimensions
5   → compute insertion location
6   → produce door object with Wall_Id

```

Door extraction depends on wall geometry being available first, which is why wall processing occurs earlier in the sequence.

Rack extraction

```

1 Load rack PLY
2   → DBSCAN clustering
3   → OBB fitting per cluster
4   → detect pillar edges
5   → identify Z-level groupings
6   → create rack + equipment representation

```

Rack extraction is important because it acts as the bridge between 3D geometry and 2D image enrichment. The rack structure created here becomes the anchor for YOLO-detected equipment.

Cabinet and battery cabinet extraction

Cabinet-oriented extractors use clustering and oriented bounding box fitting to convert irregular point clusters into structured enclosure geometry. Battery cabinets are processed similarly, but they also become parent objects for nested battery and rectifier entities.

Battery and rectifier extraction

These components are interpreted relative to their parent cabinet context. Rather than being treated as fully independent room objects, they are assigned to the nearest compatible parent based on spatial proximity and bounded geometry.

Floor and ceiling generation

Floor and ceiling are not simply copied from segmented surfaces. They are derived as structured slabs using wall boundaries, elevation references, and configurable thickness values so that the resulting geometry is consistent for IFC export.

Image-based enrichment

Post-Processing augments point-cloud geometry with image-derived detections. This is especially useful for rack internals where visual identification of mounted equipment can provide richer semantic detail than geometry alone.

```
1 image_processing_main(images_dir, data, out_path)
2 |
3 |   Load YOLO model
4 |   Iterate through extracted images
5 |     Run inference
6 |     Detect equipment classes
7 |     Map detections to rack structure
8 |   Merge results into rack Equipments[]
```

Aspect	Details
Model role	Detects rack-mounted equipment from extracted images
Typical detections	Router, switch, surge protector, patch-panel-like and related equipment classes
Integration point	Merges results into the structured rack data created during point-cloud extraction

Value added	Improves semantic richness before IFC creation
-------------	--

Conflict resolution

Because segmentation and clustering can produce overlapping or overly tight object footprints, Post-Processing includes a dedicated reconciliation phase to improve physical plausibility.

```

1 resolve_equipment_conflicts(data)
2   → detect overlaps across equipment categories
3   → apply priority rules
4   → shrink or shift lower-priority objects
5
6 resolve_equipment_wall_conflicts(data)
7   → detect wall penetrations
8   → move equipment inward
9
10 check RackEquipmentOverlapAndAdjust(Rack)
11   → evaluate vertical and internal rack overlap
12   → redistribute equipment positions

```

Function	Problem addressed	Typical outcome
<code>resolve_equipment_conflicts</code>	Object-to-object overlap between cabinets, racks, battery cabinets, and batteries	Adjusted footprints or positions based on priority
<code>resolve_equipment_wall_conflicts</code>	Equipment intruding into walls	Equipment pulled inward from wall planes
<code>check RackEquipmentOverlapAndAdjust</code>	Internal overlap of rack-mounted equipment	Z-level redistribution and cleaner rack layout

i This phase is what turns raw detections into a model that is much more suitable for downstream IFC generation and visualization.

IFC generation

Once the structured JSON has been assembled, the service delegates file creation to an embedded .NET engine. Python remains responsible for orchestration, but IFC writing is handled by the dedicated DLL.

```

1 IFCCreator.create_ifc(json_file_path)

```

```

2 |
3 | — Validate JSON exists
4 | — Build command:
5 |   ["dotnet", "IFCCreator.dll", json_file_path]
6 | — Execute subprocess
7 | — Enforce timeout
8 | — Inspect exit code
9 | — Return success/failure + output message

```

The expected output is a newly created IFC file in the working directory, typically named as a transformed version of the input JSON filename.

Structured output model

The JSON produced before IFC conversion acts as the canonical handoff contract between Python-based extraction logic and the .NET IFC creator.

```

1 | {
2 |   "wall": [{ "Wall_Id": "W0", "Type": "wall", "Height": 3.2,
3 |     "BottomFacePoints": [] }],
4 |   "door": [{ "Door_Id": "D0", "Wall_Id": "W0", "Height": 2.1,
5 |     "Length": 0.9, "Location": {} }],
6 |   "cabinet": [{ "Cabinet_Id": "C0", "Height": 2.0, "BottomFacePoints":
7 |     [] }],
8 |   "battery_cabinet": [{ "BatteryCabinet_Id": "BC0", "Height": 1.8,
9 |     "BottomFacePoints": [] }],
10 |   "rack": [{ "Rack_Id": "R0", "Height": 2.2, "BottomFacePoints": [],
11 |     "Equipments": [] }],
12 |   "battery": [{ "Battery_Id": "B0", "Parent_Id": "BC0", "Height": 0.5
13 |     }],
14 |   "rectifier": [{ "Rectifier_Id": "Rect0", "Parent_Id": "BC0",
15 |     "Height": 0.3 }],
16 |   "floor_ceiling": [{ "Unique_Id": "F0", "Type": "floor", "Points": []
17 |     }],
18 |   "project_properties": {},
19 |   "site_properties": {}
20 | }

```

Section	Purpose	Examples
Envelope geometry	Represents room shell and openings	Walls, doors, floor, ceiling
Equipment geometry	Represents major operational assets	Racks, cabinets, battery cabinets
Nested equipment	Represents child objects associated to a parent enclosure	Batteries, rectifiers, rack internals
Properties	Carries non-geometric business and site metadata	Project properties, site properties

Upload and signaling flow

```
1 IFC created
2
3   └─ Request SAS upload URL from DataRepo
4   └─ Upload IFC to DataRepo
5   └─ Upload IFC to Blob storage as backup
6   └─ Signal Root MS activity completion
7   └─ Update job state
```

Target	Purpose
DataRepo	Primary delivery location for the IFC artifact
Blob storage	Backup or internal artifact retention
Root MS	Workflow completion signal for orchestration and status tracking

Integration boundaries

Input from Segmentation

The upstream queue message is expected to include segmented class artifacts, extracted images, workflow identifiers, and metadata. Post-Processing depends on Segmentation having already completed class separation and image extraction.

```
1 {
2   "fileId": "uuid",
3   "workflowId": "workflow-id",
4   "segmentedFiles": [
5     { "class": "wall", "blobPath": "segmentation-
6 store/segmented/wall.ply" },
7     { "class": "door", "blobPath": "segmentation-
8 store/segmented/door.ply" },
9     { "class": "rack", "blobPath": "segmentation-
10 store/segmented/rack.ply" }
11   ],
12   "imageFiles": ["segmentation-store/images/img_001.png"],
13   "metadata": {
14     "customer": "customer-name",
15     "site": "site-name",
16     "project": "project-name"
17   }
18 }
```

Output to Root MS

After successful upload, the service signals completion back to the orchestration layer so the overall workflow can advance or be marked complete.

```
1 {
```

```

2  "workflowId": "workflow-id",
3  "activityName": "POST_PROCESSING",
4  "status": "COMPLETED",
5  "ifcBlobPath": "segmentation-store/ifc/file.ifc"
6  }

```

Failure handling and resilience

Scenario	Expected handling
Missing PLY input in Blob	Log the issue, skip the affected component if possible, and continue with available artifacts
No clusters returned for a component	Emit an empty object list for that class rather than failing the entire pipeline
Wall extraction failure	Use a fallback enclosure strategy where possible to support downstream geometry generation
IFC subprocess timeout	Terminate the process, mark the job as failed, and avoid indefinite runtime blocking
Primary upload failure	Retry with exponential backoff before surfacing failure
Blob upload failure	Retry using helper-based retry behavior
Message processing failure	Allow queue retry semantics to re-drive the message until retry limits are reached
YOLO model failure	Continue with point-cloud-only output if image enrichment cannot be completed

 The most critical service boundary is the transition from structured JSON to IFC generation. If the JSON is geometrically inconsistent or incomplete, downstream IFC creation can fail even when earlier extraction steps appear successful.

Operational summary

Post-Processing is the interpretation and packaging layer of the pipeline. It does not perform semantic segmentation itself, and it does not own upstream point classification. Instead, it consumes already-segmented artifacts, reconstructs usable geometry and object relationships,

enriches the model with image detections, resolves physical inconsistencies, and produces the final IFC deliverable for downstream systems.